

Resenha Crítica: Ocultação de Informação e o Fim do Fluxograma

Autor da Resenha: Bernardo Gonçalves Levy

Disciplina: Projeto de Software

Artigo Analisado: *On the Criteria To Be Used in Decomposing Systems into Modules* (D.L. Parnas, 1972)

1. Visão Geral: Repensando a Divisão de Sistemas

No artigo "On the Criteria To Be Used in Decomposing Systems into Modules", D.L. Parnas discute a modularização como um mecanismo para melhorar a flexibilidade e a compreensibilidade de um sistema, permitindo também a redução do seu tempo de desenvolvimento. A tese central do autor é que a eficácia da modularização depende totalmente dos critérios utilizados para dividir o sistema em módulos.

Para ilustrar isso, Parnas apresenta o problema de design de um sistema de produção de índice KWIC (Key Word in Context) e compara duas abordagens distintas de decomposição. A primeira reflete o método convencional, onde a divisão é feita baseada nas etapas de processamento, semelhante à criação de um fluxograma. A segunda abordagem introduz o conceito de "ocultação de informação" (information hiding), provando que esta decomposição não convencional oferece vantagens claras para os objetivos de um bom design.

2. Análise Crítica: O Erro da Abordagem Temporal

A grande contribuição de Parnas é desafiar a intuição primária de qualquer desenvolvedor. Ele afirma categoricamente que é quase sempre incorreto iniciar a decomposição de um sistema em módulos com base em um fluxograma.

Na decomposição convencional, as interfaces entre os módulos costumam ser formatos de dados e organizações de tabelas complexos. O grande perigo dessa abordagem é o acoplamento excessivo: uma única mudança em uma decisão de design (como a forma de armazenar os dados na memória) pode forçar alterações em absolutamente todos os módulos do sistema.

Em contrapartida, Parnas propõe uma inversão metodológica:

- Em vez de focar no fluxo de execução, o design deve começar com uma lista de decisões difíceis ou decisões que têm probabilidade de mudar no futuro.
- Cada módulo deve ser projetado com o objetivo expresso de ocultar essa decisão de design do restante do sistema.
- Como resultado, as interfaces tornam-se muito mais abstratas e simples — consistindo primariamente em nomes de funções e parâmetros. Isso permite que o desenvolvimento independente de cada módulo comece muito mais cedo.

Em suma, como as decisões de design transcendem o tempo de execução, os módulos não devem, sob nenhuma hipótese, corresponder a etapas cronológicas no processamento do software.

3. Aplicação Prática: Ocultando a Plataforma na Pesquisa de Mercado

Trazendo o conceito de Parnas para a minha realidade na Offerwise, programando lógicas complexas para pesquisas de mercado, os paralelos são imediatos. No desenvolvimento de questionários no Qualtrics utilizando jQuery, o instinto inicial é sempre criar um script "fluxograma": o respondente clica na tela, o script captura o evento, lê o DOM (Document Object Model) para pegar a resposta, valida a cota e oculta ou exhibe a próxima pergunta.

O problema ocorre quando o Qualtrics altera a estrutura de suas classes HTML, ou quando o cliente muda abruptamente a regra do *skip logic*. Se o código for um grande fluxograma, a manutenção se torna um pesadelo. Aplicando os critérios de Parnas:

- A ordem temporal em que o processamento ocorre não deve ser usada para definir as funções do script.
- Precisamos identificar o que "tem probabilidade de mudar" (ex: as IDs dos elementos HTML do Qualtrics e as regras demográficas do cliente).
- Devemos criar um módulo (uma classe ou conjunto de funções em JavaScript) exclusivamente para interagir com a interface do Qualtrics, ocultando totalmente a estrutura de dados e seus procedimentos de acesso.
- A lógica de negócios da pesquisa apenas "chama" essas funções abstratas. Se a plataforma for atualizada, apenas o módulo de integração do DOM é alterado, preservando toda a lógica de negócio do questionário.

4. Conclusão

O artigo de D.L. Parnas, embora escrito na década de 1970, descreve o DNA da engenharia de software moderna. Ele prova que a modularização não se trata apenas de quebrar um código grande em arquivos menores, mas sim de estabelecer fronteiras de isolamento.

No ambiente ágil da pesquisa de mercado, onde mudanças de escopo ocorrem com o questionário já em produção, esconder os detalhes de implementação das diferentes partes do código não é um luxo arquitetural, é uma necessidade de sobrevivência. O autor evidencia que um bom trabalho de decomposição, focado na ocultação de informação, não só facilita a manutenção imediata, como pode resultar em um reaproveitamento considerável do trabalho de um projeto para outro, algo essencial para escalar a programação de múltiplas pesquisas.